

Event Manager

Kotlin + Firebase Android App — Features & Architecture

Assignment: Event Management App (Kotlin + Firebase)

Package: com.techexactly.eventmanager • Language: Kotlin • UI: XML (Material Components) •
Architecture: MVVM + Repository + Hilt • Backend: Firebase (Auth, Firestore, Cloud Messaging) + an
optional Cloud Function for scheduled reminders

This revision supersedes the earlier draft: every item that draft listed under “Known limitations / next steps” has since been implemented (Section 8).

1. Overview

Event Manager is an Android app that lets a user sign up / log in with email and password, create and manage personal events (title, description, date & time, location), and see a dashboard summarizing their events, all backed by Firebase. Every event is private to the signed-in user and updates in real time across the app the moment it changes in Firestore — including changes made while offline, thanks to Firestore's persistent local cache.

The codebase follows MVVM (Model-View-ViewModel) with a Repository layer between the ViewModels and the Firebase SDK, and Hilt for dependency injection, so UI code never talks to Firebase directly and the app's core logic (validation, stats, pagination/filtering) can be unit tested without mocking Firebase or Android.

2. Architecture

2.1 Layers

The app is organized into layers, each with a single responsibility:

- **UI layer** (ui/) — Activities and Fragments render state and forward user actions to a ViewModel. They hold no business logic and no Firebase references.
- **ViewModel layer** — one ViewModel per screen (AuthViewModel, EventListViewModel, AddEditEventViewModel, DashboardViewModel). Each exposes a Kotlin StateFlow the UI collects with repeatOnLifecycle, so state survives configuration changes and never leaks updates to a dead view.
- **Data layer** (data/) — AuthRepository, EventRepository and FcmTokenRepository are the only classes that import Firebase SDK types. They expose suspend functions and Flows, and translate Firebase exceptions into plain, readable error messages before anything reaches the UI.
- **DI layer** (di/) — a Hilt module wiring the three Firebase SDK singletons (FirebaseAuth/FirebaseFirestore/FirebaseMessaging); everything else is constructor-injected directly.

2.2 Package layout

```
ui/  
  auth/      LoginActivity, SignupActivity, ForgotPasswordActivity, AuthViewModel  
  main/      MainActivity (bottom-nav shell: Events + Dashboard tabs)  
  events/    EventListFragment, EventAdapter, LoadMoreAdapter, EventListViewModel,  
             AddEditEventActivity, AddEditEventViewModel  
  dashboard/ DashboardFragment, DashboardViewModel  
di/          AppModule - Hilt bindings for FirebaseAuth / FirebaseFirestore / FirebaseMessaging  
data/  
  model/     Event, Resource<T> (Loading/Success/Error wrapper)  
  repository/ AuthRepository, EventRepository, FcmTokenRepository  
notifications/  
  EventFcmService (push, Hilt entry point), EventReminderWorker (local WorkManager reminder)  
util/  
  Validators, DateTimeUtils, EventStats, EventListPaging (all pure, unit-tested),  
  EdgeToEdge, NotificationHelper  
functions/  
  Cloud Function (Node.js): sendEventReminders, a scheduled job that pushes  
  automatic FCM reminders (optional - see Section 5.2)  
EventManagerApp.kt  enables Firestore offline persistence, creates the notification channel
```

2.3 Data flow example: adding an event

AddEditEventActivity collects form input and calls AddEditEventViewModel.save(...). The ViewModel validates the input via Validators, and on success calls EventRepository.addEvent(), which writes to users/{uid}/events/{eventId} in Firestore. Because EventListViewModel and DashboardViewModel are both listening to the same collection via a real-time snapshot listener (EventRepository.observeEvents(), wrapped in a Kotlin callbackFlow), the events list and the dashboard chart update automatically — no manual refresh, no shared ViewModel needed between screens.

2.4 Dependency injection (Hilt)

Hilt roots the dependency graph from EventManagerApp (@HiltAndroidApp) through every @AndroidEntryPoint Activity/Fragment/Service and @HiltViewModel. The only hand-written module, di/AppModule, exists purely because FirebaseAuth/FirebaseFirestore/FirebaseMessaging come from static factory functions rather than an @Inject constructor Hilt could call itself — every repository, ConnectivityObserver and ViewModel below that is constructor-injected with no extra wiring, and a test can still swap in a fake/mock repository since nothing reaches for a singleton directly.

3. Data model & storage

3.1 Firestore layout & security rules

Each user's events and registered device tokens live under their own Auth uid, so the rules stay simple ownership checks. These now live as an actual file in the repo, `firestore.rules` (deployable with `firebase deploy --only firestore:rules`), not just documentation:

```
users/{uid}/events/{eventId}
users/{uid}/fcmTokens/{token}

rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId}/events/{eventId} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }
    match /users/{userId}/fcmTokens/{tokenId} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }
  }
}
```

3.2 Event fields

Field	Type	Notes
title	String	Required; validated non-blank before save
description	String	Optional
dateTimeMillis	Long (epoch millis)	Required; validated to not be in the past
location	String	Optional
userId	String	Owner's Firebase Auth uid
createdAt	Long (epoch millis)	Set once, on creation
reminderSent	Boolean	Set by the scheduled Cloud Function once it has pushed a reminder for this event, so

Offline persistence is enabled explicitly in `EventManagerApp` via Firestore's persistent local-cache settings, so reads/writes keep working without a network connection and sync once connectivity returns.

4. Features

4.1 Authentication

- Email & password sign-up and login (Firebase Authentication).
- Password reset via a dedicated “Forgot password” screen that sends a reset email.

- Login session is persisted automatically by the Firebase Auth SDK — a returning user skips straight past the login screen.
- Firebase's typed auth exceptions (invalid credentials, invalid user, weak password, email already in use, no network) are mapped to short, human-readable error messages shown via a Snackbar.

4.2 Event management

- Add / edit / delete events with title, description, date & time, and location.
- Events list sorted in reverse-chronological order (newest date first), with an UPCOMING / PAST badge per item.
- **Pagination:** the list loads 10 events at a time with a “Load more” control at the bottom; revealing another page is instant, since it just widens the slice already streamed in by the real-time listener rather than issuing a new query.
- Real-time updates: a Firestore snapshot listener means the list (and dashboard) update instantly when an event is added, edited, or deleted — including from the offline cache before a write reaches the server.
- Input validation: title is required; date & time must be set and cannot be in the past.
- Firebase exceptions are caught at every repository call (including loading a single event to edit) and surfaced as a readable error rather than crashing.

4.3 Dashboard & statistics

- Total / upcoming / past event counts as stat cards.
- A bar chart (MPAndroidChart) of events grouped by month, built from the same real-time stream the events list uses (and unaffected by that list's pagination — the dashboard always reflects every event).
- All of the grouping/counting math lives in a pure function, `EventStats.compute()`, kept separate from the ViewModel specifically so it's unit-testable (see Section 6).

4.4 Bonus features implemented

- **Search and filter** — a toolbar `SearchView` filters the events list live by title or location, combined with All / Upcoming / Past filter chips; both compose with pagination via a single pure function, `EventListPaging.compute()` (Section 6).
- **Dark mode** — a Material3 DayNight theme with matching values-night resources for real dark surfaces (not just a dark toolbar), following the system theme automatically.
- **Push notifications** — a complete two-layer implementation, from on-device reminders to a real server-triggered push reaching any signed-in device; see Section 5.
- **Unit tests** — see Section 6.

4.5 Edge-to-edge display

Android 15+ enforces edge-to-edge display and no longer lets an app opt out, which is what made the toolbar render underneath the status bar before this was addressed. `util/EdgeToEdge.kt` centralizes the fix: every screen makes both system bars transparent and insets its own toolbar/list/buttons by the actual

bar height (via `WindowInsetsCompat`) instead of drawing content underneath them, and dark/light status-bar icon contrast is set per screen based on what's actually behind the bar.

5. Notifications: two complementary layers

“Push notification reminders for events” is implemented as two complementary pieces, since a phone app can never schedule a push to itself — sending one always requires something outside the device (a server, or a person, sending the message).

5.1 On-device reminder (works with zero setup)

EventReminderWorker uses WorkManager to schedule a local notification ~30 minutes before each event's start time. It's scheduled/rescheduled whenever an event is saved, and cancelled when the event is deleted. This works fully offline and needs no backend — it already satisfies the assignment's reminder requirement on its own.

5.2 Server-triggered push (automatic, cross-device)

The app now persists this device's FCM registration token to Firestore at `users/{uid}/fcmTokens/{token}` (FcmTokenRepository, synced on every login/session-resume in MainActivity, refreshed via `onNewToken`, and removed on logout so a shared device stops receiving a signed-out account's pushes). A scheduled Cloud Function, `functions/sendEventReminders`, runs every 5 minutes, finds events starting in the next 30 minutes that haven't been reminded yet (collectionGroup query across every user, guarded by the `reminderSent` field), and pushes a data message to every token that user has registered — via `admin.messaging().sendEachForMulticast()`, with automatic cleanup of tokens FCM reports as dead.

This is what lets a reminder reach *any* signed-in device for that account (a second phone, a reinstall, or a case where the local WorkManager job never fired), and it is a genuine server-sent push, not the client scheduling a notification to itself.

5.3 Foreground vs. background delivery

`EventFcmService.onMessageReceived` is invoked by the OS for a data message whenever the app process is alive — foreground, backgrounded, or on any screen — so there is no separate “foreground-only” code path to maintain: the app always builds and shows the notification itself through `NotificationHelper`, rather than relying on the OS's own tray display (which only auto-shows a “notification”-type payload while the app is backgrounded). That is also why the Cloud Function sends a data-only message rather than a notification-type one: it guarantees this method, and therefore consistent notification styling, fires every time, in every app state.

5.4 Deploying the Cloud Function (optional)

The app works fully without this — the on-device reminder alone satisfies the assignment. To also enable automatic cross-device pushes:

```
npm install -g firebase-tools # if not already installed
firebase login
firebase use --add            # pick the same project as your google-services.json
cd functions && npm install && cd ..
firebase deploy --only functions,firestore:rules,firestore:indexes
```

Scheduled functions require the Firebase **Blaze** (pay-as-you-go) plan to deploy at all — Cloud Scheduler's free tier easily covers a 5-minute job, so this normally costs nothing in practice. `firestore.indexes.json` already declares the composite index the query needs, so the deploy command above creates it automatically.

6. Testing

Three JVM unit test classes cover the app's testable business logic (no Android framework, no Firebase, no mocking needed):

- **ValidatorsTest** — email format, password length, password confirmation match, required event title, and “date can't be in the past” rules.
- **EventStatsTest** — total/upcoming/past counting and the month-bucketing logic behind the dashboard's bar chart, including that buckets come out in chronological order.
- **EventListPagingTest** — the events list's search-match, status-filter and “load more” page-size math: that a page is capped at the page size, that `canLoadMore` reflects whatever's left, that search matches title or location case-insensitively, and that the Upcoming/Past filters split correctly.

Run with:

```
./gradlew testDebugUnitTest
```

`Validators`, `EventStats` and `EventListPaging` are all plain Kotlin objects with no Android imports, which is what makes them safe to test directly — a deliberate design choice (see Section 2.1's theme of keeping business logic out of Android/Firebase-coupled classes).

7. Setup summary

Full step-by-step instructions are in `README.md`. In short:

- Create a Firebase project and register the app under package `com.techexactly.eventmanager`.
- Enable Authentication (Email/Password), Firestore, and Cloud Messaging.
- Download `google-services.json` from the Firebase console and place it at `app/google-services.json` (a placeholder template is at `app/google-services.sample.json` for reference — it is gitignored since it's specific to each developer's own Firebase project).
- Deploy `firestore.rules` (`firebase deploy --only firestore:rules`), or paste its contents into the console's Firestore > Rules tab.
- Open in Android Studio, let Gradle sync, and run on a device/emulator with Google Play services.
- (Optional) Deploy the Cloud Function for automatic cross-device reminders — see Section 5.4.

8. Resolved since the last revision

The previous version of this document flagged three items under “Known limitations / next steps.” All three are now implemented:

- **“FCM device tokens aren't persisted to Firestore.”** Fixed — FcmTokenRepository persists every token to users/{uid}/fcmTokens/{token}, synced on login/session-resume and on refresh, and cleaned up on logout (Section 5.2).
- **“No pagination on the events list.”** Fixed — the list now pages 10 events at a time with a “Load more” control (Section 4.2), covered by EventListPagingTest (Section 6).
- **“Automatic, scheduled push reminders would need a small server component.”** Built — functions/sendEventReminders is exactly that component, ready to deploy (Section 5.2, 5.4).

The one thing genuinely outside this project's control: a scheduled Cloud Function must actually be *deployed* by whoever owns the Firebase project (Section 5.4) — Claude/this codebase can't push it to a live project on your behalf. Everything client-side is unconditionally in place regardless of whether that deployment step is ever taken, since the on-device WorkManager reminder (Section 5.1) needs no server at all.